# Scalable distributed random number generation based on homomorphic encryption

Thanh Nguyen-Van[1,2] Tuan Nguyen-Anh[1] Tien-Dat Le[1] Minh-Phuoc Nguyen Ho[2]
Tuong Nguyen-Van[2] Quang Nhat Le[1] Khuong Nguyen-An[2,(✉)]

[1]GINAR LIMITED, *Flat/RM 2, 5/F 40 Bonham Strand, Sheung Wan, Hong Kong*
{thanhnv, tuanna, datlt, quangln}@GINAR.io
[2]Faculty of Computer Science and Engineering, University of Technology (HCMUT),
*268 Ly Thuong Kiet Street, District 10, Ho Chi Minh City, Vietnam*
{1513056, 1612736, 1614028, nakhuong}@hcmut.edu.vn

***Abstract*— Generating a secure source of publicly-verifiable randomness could be the single most fundamental technical challenge on a distributed network, especially in the blockchain context. Many current proposals face serious problems of scalability and security issues. We present a protocol which can be implemented on a blockchain that ensures unpredictable, tamper-resistant, scalable and publicly-verifiable outcomes. The main building blocks of our protocol are homomorphic encryption (HE) and verifiable random functions (VRF). The use of homomorphic encryption enables mathematical operations to be performed on encrypted data, to ensure no one knows the outcome prior to being generated. The protocol requires $\mathcal{O}(n)$ elliptic curve multiplications and additions as well as $\mathcal{O}(n)$ signature signing and verification operations, which permits great scalability. We present a comparison between recent approaches to the generation of random beacons.**

***Index Terms*—random beacons, homomorphic encryption, elliptic curve cryptography, verifiable random function, distributed ledgers, decentralized schemes, blockchain-based applications.**

## I. Introduction

In cryptography, the security strength of a system can be evaluated via the quality of random numbers it uses. The degree of randomness required varies according to which context it is used for. Although randomness seems very naturally obvious, creating it is surprisingly challenging. A random number generator (RNG), in general, must satisfy a basic set of criteria, which includes *uniform distribution* and (strict) *unpredictability*. However, many applications today require random numbers to be not only unpredictable but also *publicly-verifiable* and *tamper-resistant* and does not rely on any single party. For example, blockchain-based contracts often need to generate random numbers for use cases such as lotteries. Traditionally, lottery games use a public randomness source such as ball machines to prove that the winning numbers are drawn at random. The process of choosing the winning numbers may appear random but it is believed that there are people who can manipulate the result of a lottery game (see *Pennsylvania Lottery scandal*, https://bit.ly/2TXTKZq). The ball machines were designed by the game operators (or their partners) and it is still not known if any backdoor has been attached to the machines. Therefore, there are chances that this randomness source has been tampered with. Hence, it is demanding to construct an RNG that produces publicly-verifiable, tamper-resistant and fair outcomes for the use on blockchain.

Since Nakamoto introduced Bitcoin [1], the very first application of the *blockchain technology* (*blockchain* for short), more and more attention has been paid to this promising technology. Indeed, blockchain offers many interesting properties [1] (transparency, verifiability, tamper-resistance, etc.) that enable us to address the above-mentioned problem. Afterwards, there has been a substantial amount of effort in constructing the so-called distributed random number generator (dRNG) that runs on a distributed network such as a blockchain. In this paper, we propose a protocol for constructing such a dRNG.

***Our contributions:*** Our protocol is essentially concerned with the problem of verifiability and trustworthiness, rather than the level of randomness as a whole. At a high level, the protocol enables many parties on a distributed to co-operate to generate a random number. The protocol is divided into rounds with respect to a **Requester**. In each round, each party runs an instance of a VRF to determine if he is eligible to become a contributor. Afterwards, a contributor encrypts his contribution using the public key of the **Requester** before publicizing it. We take advantage of the homomorphic property of the ElGamal cryptosystem on elliptic curve [2] to make it possible for anyone to calculate the sum of all encrypted contributions without performing any decryption. Finally, the **Requester** uses his private key to decrypt the tallied contribution once and gets the random beacon. He also generates a proof of proper decryption to prove that the random beacon was properly decrypted form the tallied contribution. Each party is required to generate a proof for each contribution. We utilize digital signatures, cryptographic hash functions and zero-knowledge proofs [3] to allow any external verifier to check the eligibility and integrity of a contributed value. Our protocol achieves the following:

1) **Unpredictability:** No party is able to learn anything about the generated number prior to being revealed.
2) **Public Verifiability:** With publicized information, anyone can attest the correctness of a generated random number.
3) **Tamper-resistance:** A number once generated, is cor-

978-1-7281-4693-5/19/$31.00 ©2019 IEEE
DOI 10.1109/Blockchain.2019.00083

rectly formed by valid contributions.

4) **Scalability:** The protocol has the overall complexity of $\mathcal{O}(n)$, where $n$ is the number of parties on the network.
5) **Honest minority:** If there is at least one honest contributor, the generated number is uniformly distributed.

***Structure of the paper:*** Section II explores recently proposed algorithms. Section III recalls some background knowledge of the main building blocks used in our protocol. Section IV describes the detailed protocol. The security and complexity analyses can be found in Sections V, VII, respectively. We also present a comparison of different current solutions to random beacons in Section VIII. Finally, Section IX concludes this paper.

## II. Related Works

Generation of public randomness has been examined in a wide range of contexts. In this section, we go through some of the recent methods to clarify the current state of the problem.

In [4], the authors used Bitcoin as a public randomness source. However, this approach is not bias-resistant due to the fact that miners can manipulate the block hash.

In 2017, Cascudo and David [5] presented *SCRAPE* which utilizes Schoenmakers' secret sharing protocol [6]. SCRAPE protocol requires a distributed ledger for exchanging information between parties. Each party chooses a number and then use a publicly verifiable secret sharing (PVSS) protocol to share his secret to the others. After that, parties reveal their respective secrets and all revealed secrets are combine to form the random beacon. If a party fails to reveal his secret, a PVSS's recovery procedure will be run to reconstruct this secret. The protocol is not effectively scalable due to its communication complexity of $\mathcal{O}(n^3)$.

In an orthogonal work, Syta et al. [7] introduced the *Rand* protocol family including: *RandShare, RandHound, RandHerd*. These protocols employ PVSS in a Byzantine fault tolerant (BFT) [8] context. RandShare uses the same technique as in SCRAPE and offers complexity of $\mathcal{O}(n^3)$. RandHound and RandHerd are also based on PVSS but they operate slightly different. In RandHound, a client divides servers into several smaller groups and combines the random inputs from these groups to generate the random beacon. RandHerd's design is based on RandHound, and a $(t, n)-$threshold Schnorr signature (TSS) scheme [9]. Additionally, the protocol leverages on *collective signing* (CoSi) [7] to generate a sequence of random numbers. The authors stated that the complexity of RandHound and RandHerd are $\mathcal{O}(c^2 n)$ and $\mathcal{O}(c^2 \log n)$, respectively (where $c$ a the group size). However, according to our knowledge, $c$ is dependent on $n$ (see [10]) and is therefore not a constant.

HydRand [10] is also a PVSS-based protocol geared towards the continuous provision of randomness in a Byzantine failure setting. The protocol mainly operates through four steps: 1) a leader is chosen based on the random number from the previous round; 2) the leader reveals his previously shared secret and then chooses a random number and runs PVSS protocol to commit a new random secret; 3) all other nodes run an instance of Byzantine agreement to agree on the content of the received messages; 4) after some specified rounds, the leader will reveal his secret. If the leader fails to do this, other nodes start to run the PVSS's recovery procedure to obtain the random number. Since the protocol only requires to run PVSS once, therefore, the complexity is reduced from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$. However, this complexity is still too high for a system to be scalable.

In [11], Dfinity aims at constructing a verifiable random function as a building block for reaching consensus on a permissioned network. Dfinity obtains random beacon by utilizing BLS signatures [12]. The randomness generation process consists of 1) a one-time setup in which a Distributed Key Generation is run; and 2) a repeated signing process in which the outputs are produced. Although the Distributed Key Generation is slow, the repeated signing process is much faster. Overall, the protocol offers the complexity of $\mathcal{O}(cn)$ (the setup complexity is not included).

In an another approach, RanDAO [13] has deployed an RNG based on a commit-reveal scheme on the Ethereum blockchain. The main idea is that all parties 1) choose a secret of their own and commit to their secret (by submitting a hash of the secret); 2) reveal their secret; 3) and combine all the secrets to obtain the random number. The protocol assumes that there is at least one honest party (so-called *honest minority)* for the outcome remains unpredictable. The protocol's complexity is only $\mathcal{O}(n)$, which offers great scalability. However, the main problem of this protocol is that the last person to reveal the secret will be able to know the resulting number beforehand and this person can refuse to reveal the secret value leading the protocol to termination without producing any outcome.

## III. Backgrounds

In this section, we recall some notions and basic results which are the building blocks of our protocol.

### A. Homomorphic Encryption

Assume that the plaintext space forms a group $(P; \diamond)$ and the ciphertext space forms a group $(C; \circ)$, then the encryption algorithm $E$ is a map from the group $P$ to the group $C$, i.e., $E_k : P \longrightarrow C$, where $k \in K$ is either a secret key (in a secret key cryptosystem) or a public key (in a public-key cryptosystem). For all $a$ and $b$ in $P$ and $k$ in $K$, if $E_k(a) \circ E_k(b) = E_k(a \diamond b)$ the encryption scheme is **homomorphic.**

Homomorphic encryption can be classified into three types: **partially homomorphic** one which only allows addition or multiplication (RSA [14], El-Gamal [15], Paillier [16], etc.); **somewhat homomorphic** one which allows both addition and multiplication with limited times, such as Boneh-Goh-Nissim encryption scheme [12]; **fully homomorphic** one which allows both addition and multiplication to be performed unlimited times (see [17]).

Regarding performance aspect, the study in [18] shows that somewhat homomorphic and fully homomorphic encryption schemes are not very practical while partially homomorphic

encryption schemes have already been widely used. Moreover, the scope of this paper only requires the homomorphic property for one operation. Hence, partially homomorphic encryption is a good-enough choice.

In this paper, we consider an elliptic curve $\mathcal{E}$ over a finite field $\mathbb{F}_p$ ($p \neq 2, 3$) for the cyclic group in the El-Gamal encryption scheme, since it offers smaller keys compared to non-EC cryptography at the same level of security (see [19]). Its security depends on the difficulty of a problem in that group related to the hardness of the elliptic curve discrete logarithm problem (ECDLP) [3].

Given an elliptic curve $\mathcal{E}$ with a generator $\mathcal{G}$ of order $N$ over a finite field $\mathbb{F}_p$ with $p$ prime. The El-Gamal cryptosystem consists of the following components:

- ***Key Generation**:* Alice chooses an integer $x$ as the private key and calculate $Y = x\mathcal{G}$. Alice then publishes $Y$ as the public key.
- ***Encryption**:* to encrypt a point $M$, Bob has to
  1) choose a random integer $k \in \{0, 1, \ldots, N-1\}$.
  2) calculate $C = k\mathcal{G}$, $D = kY + M$. The ciphertext is $E(m) = (C, D)$.
- ***Decryption**:* to decrypt a ciphertext $E(m) = (C, D)$, Alice has to
  1) calculate $C' = xC$.
  2) calculate $M = D - C'$.

**Homomorphic Property:** El-Gamal encryption on elliptic curves has a homomorphic property, which allows multiple ciphertexts of the same key pair $(x, Y)$ to be added together. Indeed,

$$\begin{aligned} E(M_1) + E(M_2) &= (C_1, D_1) + (C_2, D_2) \\ &= (k_1\mathcal{G}, k_1Y + M_1) + (k_2\mathcal{G}, k_2Y + M_2) \\ &= ((k_1 + k_2)\mathcal{G}, (k_1 + k_2)Y + (M_1 + M_2)) \\ &= E(M_1 + M_2). \end{aligned}$$

### B. Zero-Knowledge Proofs

A **zero-knowledge proof** (ZKP) [3] is a method by which one party can prove to another party that he knows a value $x$, without revealing any information about $x$. Recall from Subsection III-A that the encryption of a message $M$ with public key $Y = x\mathcal{G}$ is a pair of $C, D) = (k\mathcal{G}, kY + M)$. A proof-of-proper decryption, saying that $(C, D)$ decrypts to $M$, can be constructed by anyone who knows the secret key $x$. Since $Y = x\mathcal{G}$ and $D - M = kY = kx\mathcal{G} = xC$, the proof constructed should prove the following

$$dlog_{\mathcal{G}}(Y) = dlog_C(D - M).$$

Suppose that Alice receives a ciphertext $(C, D)$ and uses her secret key $x$ to decrypt this ciphertext to get the message $M$. She wants to prove that the ciphertext $(C, D)$ properly decrypts to $M$ but does not want to reveal $x$. Alice can use the Algorithm 1 to compute the proof (note that $H$ is a cryptographic hash function [3]).

**Algorithm 1** *ZKP*$(x, Y, M, C, D)$

1: $D' \leftarrow D - M$
2: $r \xleftarrow{\$} \{0, 1, \ldots, N-1\}$, $B_0 \leftarrow r\mathcal{G}$, $B_1 \leftarrow rC$
3: $c \leftarrow H(Y, C, D', B_0, B_1)$
4: $z \leftarrow r + cx$
5: **return** $\pi(M) = (c, z)$

**Algorithm 2** *VerifyZKP*$(Y, M, C, D)$

1: $B_0 \leftarrow z\mathcal{G} - cY$
2: $B_1 \leftarrow zC - cD'$
3: **return** $c \stackrel{?}{=} H(Y, C, D', B_0, B_1)$

The proof is $\pi(M) = (c, z)$. Anyone can check that $M$ has been decrypted from $(C, D)$ using the *VerifyZKP* algorithm (Algorithm 2).

The proof has the following properties:

- **Non-interactive:** no interaction is necessary between prover and verifier.
- **Completeness:** if Alice knows $x$ and provides a proper proof $\pi(M)$, a verifier will eventually be convinced that Alice knows $x$ with high probability.
- **Soundness:** Alice can *only* convince a verifier if Alice knows $x$.
- **Zero Knowledge:** A verifier does not learn anything about $x$.

### C. Verifiable Random Functions

A verifiable random functions (VRF) is a pseudo-random function which, given an input $x$, produces a random number $y$ along with a proof $\pi$ for $y$. Using $\pi$, everyone can verify whether $y$ was generated correctly from $x$ or not. The formal definition of VRFs is as follows (see [20]).

***Definition** 1 (VRF):* A function family

$$F_{(.)}(.) : \{0, 1\}^{\ell} \mapsto \{0, 1\}^{\ell_{VRF}}$$

is called a **family of VRFs** if there exist algorithms $(Gen, Prove, Ver)$ such that:

- $Gen(1^k)$ outputs a pair of keys $(pk, sk)$;
- $Prove_{sk}(x)$ outputs a pair $(F_{sk}(x), \pi_{sk}(x))$, where $F_{sk}(x)$ is the function value and $\pi_{sk}(x)$ is the proof of correctness; and
- $Ver_{pk}(x, y, \pi)$ verifies $y = F_{sk}(x)$ using the proof $\pi$.

Additionally, we require the following properties for a family of VRFs.

1) **Uniqueness**: no values $(pk, x, y_1, y_2 \neq y_1, \pi_1, \pi_2)$ can satisfy $Ver_{pk}(x, y_1, \pi_1) = Ver_{pk}(x, y_2, \pi_2)$.
2) **Provability**: if $(y, \pi) = Prove_{sk}(x)$, then $Ver_{pk}(x, y, \pi) = 1$.
3) **Pseudo-randomness**: the output $y$ is indistinguishable from random, except with a negligible probability.

## IV. The Proposed Protocol

There are three main components of the protocol:

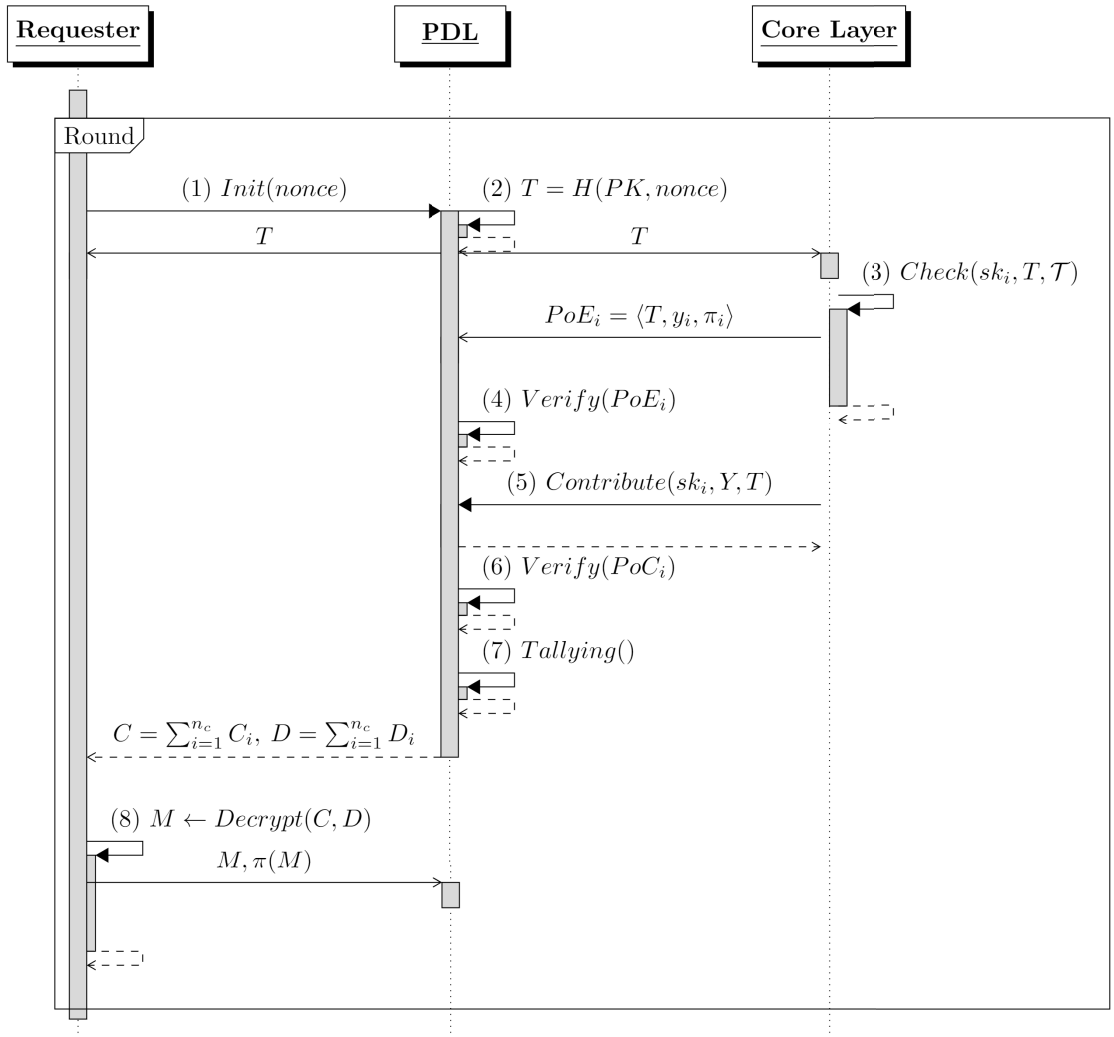


Fig. 1. Sequence diagram of the proposed protocol

- **Requester**: a client who wants to use the random number generated by our protocol.
- **Core Layer:** a network consisting of many parties which is responsible for generating random numbers.
- **Public Distributed Ledger ($PDL$):** an append-only ledger where everyone is able to see all public information including public keys, the list of contributions, etc.

Figure 1 presents the overview of how the protocol works. For the sake of simplicity, we assume that all public information (public key of the **Requester** and the parties) has been made public. Each round of the protocol simply operates through 8 stages:

1) The **Requester** sends a $nonce$ to the $PDL$ to initialize his request.
2) A ticket $T$ with respect to the **Requester** and the received $nonce$ is computed. This ticket is used to determined which parties are eligible (these are called **contributors**) to contribute to generating random numbers. This stage can be performed by running a smart contract on the $PDL$.
3) On input the ticket $T$, each party runs an instance of VRF functions to check if he is eligible for that ticket and sends a ***Proof-of-Eligibility*** ($PoE$) to the $PDL$. The $PoE$ contains sufficient information for verifying the eligibility of a party (described shortly).
4) The $PDL$ verifies all received $PoE_i$.
5) Eligible parties, before sending their contribution, encrypt it with the public key of the **Requester**. There also generate a ***Proof-of-Contribution*** ($PoC$). The protocol employs El-Gamal encryption (detail in section III) which offers many advantages:
   - First, we can compute the sum of contributions without performing any decryption.

- This is convenient for the **Requester** since he only needs to perform decryption once.
- The use of encryption helps prevent parties from seeing the contribution of others. This ensures the privacy of each contribution. Bias-resistance is also guaranteed since a party is not able to know the contribution of others, and thus cannot affect the outcome in a meaningful way.

6) The $PDL$ verifies all received $PoC_i$.
7) All the contributions are then added together and the final sum is sent to the **Requester**.
8) The **Requester** uses his secret key $x$ to decrypt the tallied contribution and publish this value along with a proof of proper decrytion.

For the rest of this paper, we denote by $E(\cdot)$ the encryption algorithm of the El-Gamal encryption scheme on an elliptic curve $\mathcal{E}$ over a finite field $\mathbb{F}_p$. Denote by $\mathcal{G}$ a generator of $\mathcal{E}$ of order $N$. $x$ and $Y = x\mathcal{G}$, respectively, are the private key and public key of the **Requester** used in El-Gamal encryption. $H(\cdot)$ denotes a cryptographic hash function [3]. We also denote by $SIG(\cdot,\cdot)$, $V(\cdot,\cdot)$ the signing algorithm and verification algorithm, respectively, of a digital signature [3].

Our protocol makes two following assumptions:

- There is at least one of the contributor who is honest. *Honest* means that the contributor follows the protocol, does not share his contribution with anyone else and chooses his contribution in a uniform way.
- The **Requester** does not share his private key $x$ with anyone else.

Each round of the protocol is divided into stages. Next, we present the detail of each stage.

### A. Initialization

At the beginning of each round, the **Requester** chooses a number (called *nonce*) and publishes it to initialize his request. The ticket is then calculated as the hash of the public key of the **Requester** and the *nonce*

$$T \leftarrow H(PK, nonce).$$

### B. Contributors Selection

In this stage, all parties use the VRF to determine their own eligibility. Our protocol specifies a *threshold* $\mathcal{T}$ to determine the expected number of parties selected for a ticket. In section VI, we will discuss in more detail about how to choose a proper threshold for a specific expected number of contributors. Following the *CheckEligibility* algorithm (Algorithm 3), each party uses the ticket $T$ as the input of the VRF, compares the output of the VRF to the threshold to check whether he is eligible or not. If eligible, the party will have to generate a $PoE_i = \langle T, y_i, \pi_i \rangle$ as a proof for their eligibility and publish this proof onto the $PDL$.

Using a VRF provides three interesting properties:

- Given a random input, the VRF produces a pseudo-random output, which is considered to be uniformly distributed between 0 and $2^{\ell_{VRF}} - 1$ where $\ell_{VRF}$ is the length of the output. Therefore, we can consider that a party is selected uniformly with respect to the ticket $T$.
- An adversary does not know the secret key $sk_i$ cannot determine if the party $i$ is eligible or not.
- The output of a VRF is publicly-verifiable.

**Algorithm 3** *CheckEligibility*$(sk_i, T, \mathcal{T})$

$y_i, \pi_i \leftarrow Prove_{sk_i}(T)$
**return** $y_i \overset{?}{<} \mathcal{T}$

### C. PoE Verification

**Algorithm 4** *VerifyEligibility*$(pk_i, T, y_i, \pi_i, \mathcal{T})$

**return** $(Ver_{pk_i}(T, y_i, \pi_i) = 1$ **and** $y_i < \mathcal{T})$

This stage can be done by anyone since each $PoE_i$ and the corresponding public key $pk_i$ have already made public. One can follow the instruction in the Algorithm 4 to check the correctness of a $PoE$.

### D. Contribution Making

Without loss of generality, assume that at this point, we have a list of $n_c$ contributors labeled from 1 to $n_c$. A contributor runs the *Contribute* algorithm (Algorithm 5). Simply put, a contributor $P_i$ has to:

**Algorithm 5** *Contribute*$(sk_i, Y, T)$

1: $x_i \leftarrow \{0, 1, \ldots, N-1\},\ M_i \leftarrow x_i\mathcal{G}$
2: $k_i \xleftarrow{\$} \{0, 1, \ldots, N-1\}$
3: $E(M_i) = (C_i, D_i) \leftarrow (k_i\mathcal{G}, k_iY + M_i)$
4: $\sigma_i \leftarrow SIG(sk_i, H(C_i, D_i))$
5: **return** $PoC_i = \langle T, C_i, D_i, \sigma_i \rangle$

- choose a value $M_i$ to contribute;
- use the public key of the **Requester** to encrypt $M_i$;
- sign his contribution with his secret key $sk_i$;
- generate a proof for his contribution $PoC$ as described in line 5 of the algorithm.

### E. Contribution Verification

This stage only checks the origin and integrity of a $PoC$, a verifier extracts $(C_i, D_i)$ from $PoC_i$ and $pk_i$ of the contributor $P_i$. Then he uses the verification algorithm $V(\cdot)$ to check the validity of the $PoC_i$. We assume that the digital signature scheme is secure, and thus, no one can construct a valid $PoC_i'$ from $PoC_i$ without having the secret key $sk_i$.

### F. Contribution Tallying

After all contributions have been made public, one can easily tally them by calculating

$$C = \sum_{i=1}^{n_c} C_i, \quad \text{and} \quad D = \sum_{i=1}^{n_c} D_i.$$

Then $(C, D)$ is sent to the **Requester**.

### G. Result Revealing

Since the final tallied result $(C, D)$ exists in the form of a ciphertext, the **Requester** uses his private key $x$ to decrypt the ciphertext. The homomorphic property of El-Gamal encryption (see Section III) gives us the following

$$M = D - xC = \sum_{i=1}^{n_c}(k_iY + M_i - xk_i\mathcal{G}) = \sum_{i=1}^{n_c} M_i.$$

In addition to $M$, the **Requester** computes a proof $\pi(M) = (c, z)$ following the Algorithm 1 to prove that $(C, D)$ decrypts to $M$. The **Requester** then publishes $M$ as the random beacon and $(c, z)$ as the proof for $M$.

## V. Security Analysis

In this section, we provide a theoretical security analysis of our protocol. Our system is built upon El-Gamal cryptosystem and some other primitives in cryptography, and they have widely accepted proofs.

### A. Honest Minority

As we can see from the previous section that the final result is indeed the sum of many points on the elliptic curve $\mathcal{E}$, especifically,

$$M = \sum_{i=1}^{n_c} M_i = (x_1 + x_2 + \cdots + x_k)\mathcal{G}.$$

We first need the following technical result.

***Lemma** 2:* If $X$ and $Y$ are two independent random variables over $\{0, 1, \ldots, N-1\}$ with $X$ uniformly distributed, the random variable $Z = X + Y \mod N$ is also uniformly distributed.

***Proof.*** Notice that $x + y = z \mod N$ if and only if $x = z - y \mod N$ (since $-y = N - y$). Hence, for each $z \in \{0, 1, \ldots, N-1\}$, by the independence of $X$ and $Y$, we have

$$\text{Prob}[X + Y = z] = \frac{1}{N}.$$

Therefore, $Z$ is uniformly distributed. ■

***Theorem** 3 (Honest minority):* If there is at least one honest contributor, $M$ is uniformly distributed over $\mathcal{E}$.

***Proof.*** Without loss of generality, assume that $P_1$ is honest and $P_1$ chooses $M_1$ randomly over $\mathcal{E}$. Also assume that an adversary $\mathcal{A}$ is able to learn all other $M_i$'s but has no knowledge of $M_1$ (this implies that $\mathcal{A}$ has no knowledge about $x_1$). In other word, $M_1$ appears to be uniformly distributed among $\mathcal{E}$ to $\mathcal{A}$. Hence, $x_1$ is uniformly distributed over $\{0, 1, \ldots, N-1\}$ with respect to $\mathcal{A}$. the probability that $\mathcal{A}$ can guess $M_1$ is the same as the probability that $\mathcal{A}$ can guess $x_1$ which is exactly $\frac{1}{N}$. To successfully obtain the value of $M$, $\mathcal{A}$ has to obtain $M_1$.

Let $x_{\mathcal{A}} = x_2 + x_3 + \cdots + x_{n_c}$, then

$$M = M_1 + x_{\mathcal{A}}\mathcal{G} = (x_1 + x_{\mathcal{A}})\mathcal{G}.$$

Since $x_1$ is uniformly distributed over $\{0, 1, \ldots, N-1\}$, from Lemma 2, we have that $(x_1 + x_{\mathcal{A}})$ is uniformly distributed over $\{0, 1, \ldots, N-1\}$. Therefore, $M$ is uniformly distributed over $\mathcal{E}$. ■

### B. Unpredictability

Our protocol assumes honest minority between contributors. Follow from Theorem 3, we can see directly that an adversary can only guess the correct value of the random beacon with a probability at most $\frac{1}{N}$. Therefore, unpredictability is ensured. We have the following:

***Theorem** 4 (Unpredictability):* Under the two above assumptions, no one is able to learn anything about the final outcome, except with a negligible probability, until it is revealed.

### C. Unbiasability

In our protocol, unbiasability follows from unpredictability and the fact that we hide the content of the contributions by employing encryption. Parties are not able to know what others contribute, and thus cannot bias the outcome in a meaningful way. Moreover, a contribution once made, cannot be withdrawn. Therefore, at the time of all contributions have been publicized, the protocol still operates without the involvement of the contributors. As a result, bias-resistance is ensured.

### D. Tamper-Resistance

***Theorem** 5 (Tamper-Resistance):* If a contributor does not share his private key with anyone else, his contribution is secured after it has been made.

***Proof.*** As we require that each contributor has to publicize a $PoC$ associated with his contribution. This $PoC$ contains a signature of the contribution that the contributor has made. Everyone can check if the contribution has been modified or not:

- The owner of the contribution can do this by comparing the publicized contribution written on the $PDL$ with the original contribution that he has in his local computer.
- Everyone else can do this by verifying the signature of the contribution. This process is feasible since the public key of the contributor has been written on the $PDL$.

Therefore, if any changes have been made to a contribution, they will be detected which implies any contribution is secured after being made. ■

### E. Public Verifiability

***Theorem** 6 (Public-Verifiability):* Any external verifier can check if a random beacon $M$ was correctly generated with complexity of $\mathcal{O}(n)$.

***Proof.*** In our protocol, a generated beacon is said to be *correct* if it is calculated from all valid contributions. A contribution is said to be *valid* if it comes from an eligible party, and it has not been tampered with.

As all $PoE$'s and $PoC$'s have been published on the $PDL$. A verifier can re-execute the stages IV-C, IV-E, IV-F to check if $(C, D)$ has been computed from valid contributions. As there

are at most $n$ $PoE$'s and $n$ $PoC$'s, the verifier only needs to do at most $2n$ verification opeartions.

Afterwards, using the proof $\pi(M)$ that the **Requester** has publicized, the verifier can check if $M$ has been correctly decrypted from $(C, D)$. ■

## VI. Choosing The Threshold

In this section, we explain how to choose the threshold $\mathcal{T}$ when we want to maximize the probability that there are exactly $k$ out of $n$ parties on the network are selected as contributors. Let $\ell_{VRF}$ be the length of the output of a VRF. Due to the pseudo-randomness property of the VRF, the probability that a party is eligible for a ticket is $p = \frac{\mathcal{T}}{2^{\ell_{VRF}}}$ and this probability is the same for all parties. In fact, if we let $X$ be the number of parties eligible for a ticket, then $X$ is a random variable following the Binomial distribution. The probability that exactly $k$ out of n parties are selected is $\text{Prob}[X = k] = B(k; n, p) = \binom{n}{k} p^k (1-p)^k$. As we all know that, when $k = mode(X) = \lfloor (n+1)p \rfloor$, $\text{Prob}[X = k]$ will be maximum. This implies that

$$\frac{k-1}{n+1} < p \leq \frac{k}{n+1}.$$

Hence, the value $\mathcal{T}$ that maximizes the probability that there are $k$ out of $n$ parties are selected should satisfy the following condition

$$\frac{k-1}{n+1} 2^{\ell_{VRF}} < \mathcal{T} \leq \frac{k}{n+1} 2^{\ell_{VRF}}.$$

Furthermore, the probability that there is no contributor for a ticket equals $\text{Prob}[X = 0] = (1-p)^n$. This probability quickly converges to zero when the number of participant increases. Therefore, with properly chosen $\mathcal{T}$, we can control the probability that a ticket has no contributors (e.g a ticket with no outcome).

## VII. Complexity analysis

In this section, we discuss the efficiency of our proposed protocol. We present the concrete computational complexity of our protocol by computing the numbers of elliptic curve point additions ($Add$) and multiplications ($Mult$) as well as the number of signing ($Sign$) and verification ($Verify$) operations in each stage. The encryption algorithm requires 1 $Add$ and 2 $Mult$'s, while the decryption algorithm requires 1 $Add$ and 1 $Mult$. The detail of the number of operations required for each stage is presented in Table I. From the table, there are $3n_c+1$ additions, $3n_c+1$ multiplications and at most $2n_p$ signing and verification operations. In theorem 6, we have also examined the verification complexity of our protocol which equals $\mathcal{O}(n)$. In conclusion, our protocol achieves the overall complexity of $\mathcal{O}(n)$, which offers great scalability.

TABLE I
Concrete computational complexity of our protocol

| Stage | Operation | | | |
|---|---|---|---|---|
| | $Add$ | $Mult$ | $Sign/Prove$ | $Verify$ |
| IV-B | – | – | $n_p$ | – |
| IV-C | – | – | – | $n_p$ |
| IV-D | $n_c$ | $3n_c$ | $n_c$ | – |
| IV-E | – | – | – | $n_c$ |
| IV-F | $2n_c$ | – | – | – |
| IV-G | 1 | 1 | – | – |

## VIII. Comparison of dRNGs

This section provides a comparison of the characteristics between different dRNG protocols. In [10], the authors have also provided a comparison between current approaches to generating randomness beacon in great detail. We only focus on the properties that a dRNG is required to possess, the cryptographic primitives each protocol uses, the complexity model, and the assumption on the number of honest nodes each protocol makes. Table II presents an overview of the comparison.

For the sake of notations, the symbol "✓" describes that a property is fulfilled or something is required (according to the context), "✗" refers to the opposition, whereas subscripts (1 - 6) explain further detail of the properties being analyzed. We denote by $n$ the number of participants in the network, $c$ is the size of subset in specific protocol ($c$ may vary between different protocols), $t$ is the least number of honest parties, and $f$ is the maximum number of byzantine parties allowed in each protocol. Addition to Table II, we provide some analyses in the following:

1) We cannot compare PoW to others in a meaningful way due to its computationally-expensive performance.
2) To ensure the availability of the protocol, the constant $c$ in the Rand protocol family [7] is actually dependent on $n$ and is therefore not a constant (see [10]).
3) In PoW, unpredictability is only achieved with probabilistic guaranties. Additionally, a miner can actually refuse to broadcast a block if he is incentivized or motivated to do so, thus bias-resistance is not guaranteed.
4) In HydRand, unpredictability is fully achieved only after $f+1$ rounds in the future.
5) According to the authors, each group is *honest* if the group size $n_{group} > 2f_{group}$ [11, Section 4.2]. The authors also assumed that all groups are honest. Hence, the number of honest parties is $t > n/2$.
6) In a PoW blockchain, we rarely consider the number of honest parties but the proportion of computational power that all honest parties possess instead. It is widely believed that if someone possesses more than half of the computational power of the blockchain, he can manipulate or revert the blockchain. Therefore, we can assume the computational power of all honest parties is more than half of the blockchain.

## IX. Conclusions

We have proposed a protocol that allows many people to cooperate to generate random numbers in a secure and verifiable way. It is designed as a stand-alone protocol with

TABLE II
COMPARISON OF DISTRIBUTED RANDOM NUMBER GENERATION PROTOCOLS

| | *Unpredictability* | *Bias-Resistance* | *Public Verifiability* | *Public Ledger* | *Communication Complexity (Overall System)* | *Computation Complexity (per Participant)* | *Verification Complexity (per Verifier)* | *Honest Participants* | *Cryptographic Primitive(s)* |
|---|---|---|---|---|---|---|---|---|---|
| RanDAO | ✓ | ✗ | ✓ | ✓ | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ | $\mathcal{O}(n)$ | $t \geq 1$ | Commit-Reveal |
| Dfinity | ✓ | ✗ | ✓ | ✗ | $\mathcal{O}(cn)$ | $\mathcal{O}(c)$ | $\mathcal{O}(1)$ | $t > n/2_{(5)}$ | BLS Signature |
| Scrape | ✓ | ✗ | ✓ | ✓ | $\mathcal{O}(n^3)$ | $\mathcal{O}(n^2)$ | $\mathcal{O}(n^2)$ | $t > n/2$ | PVSS |
| PoW | ✓$_{(3)}$ | ✗$_{(3)}$ | ✓ | ✓ | $\mathcal{O}(n)$ | very high$_{(1)}$ | $\mathcal{O}(1)$ | $t > n/2_{(6)}$ | Hash Function |
| RandShare | ✓ | ✓ | ✓ | ✗ | $\mathcal{O}(n^3)$ | $\mathcal{O}(n^3)$ | $\mathcal{O}(n^3)$ | $t > 2n/3$ | PVSS |
| RandHound | ✓ | ✓ | ✓ | ✗ | $\mathcal{O}(c^2 n)_{(2)}$ | $\mathcal{O}(c^2 n)$ | $\mathcal{O}(c^2 n)$ | $t > 2n/3$ | PVSS & CoSi |
| RandHerd | ✓ | ✓ | ✓ | ✗ | $\mathcal{O}(c^2 \log n)_{(2)}$ | $\mathcal{O}(c^2 \log n)$ | $\mathcal{O}(1)$ | $t > 2n/3$ | PVSS & CoSi |
| HydRand | ✓$_{(4)}$ | ✓ | ✓ | ✗ | $\mathcal{O}(n^2)$ | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | $t > 2n/3$ | PVSS |
| Our Protocol | ✓ | ✓ | ✓ | ✓ | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ | $\mathcal{O}(n)$ | $t \geq 1$ | HE & VRF |

its applicability in the context of blockchain, one of the most notable distributed environments.

The key technique that sets us apart from other approaches is that we use homomorphic encryption (El-Gamal cryptosystem on elliptic curves) to force contributors to encrypt their contribution before submission and it will remain encrypted until the protocol is finished. The interesting thing is that, homomorphic encryption allows us to perform addition on encrypted data without performing any decryption.

The integrity of each contribution is guaranteed and can be verified at all times without revealing it. The protocol ensures that any external verifier can check the correctness of the generation process with enough public information. The protocol assumes *honest minority* between contributors which is practical in many distributed networks. The security and complexity analyses show that our protocol can be implemented for practical use, especially in the blockchain environment. We have also presented a brief comparison between current approaches to generating random beacons.

The only limitation of our protocol is that we make an assumption that the **Requester** does not share his private key with anyone else to make the protocol secure. However, the **Requester** may collude with some contributors in the network to get the number that himself can benefit from. In the future, we will address this problem and potentially remove the involvement of the requester in the result-decryption process.

## REFERENCES


[1] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.

[2] Rosy Sunuwar and Suraj Ketan Samal. Elgamal encryption using elliptic curve cryptography. *Cryptography and Computer Security, University of Nebraska, Lincoln*, 2015.

[3] Wade Trappe. *Introduction to cryptography with coding theory*. Pearson Education India, 2006.

[4] Marcin Andrychowicz and Stefan Dziembowski. Distributed cryptography based on the proofs of work. *IACR Cryptology ePrint Archive*, 2014:796, 2014.

[5] Ignacio Cascudo and Bernardo David. Scrape: Scalable randomness attested by public entities. In *International Conference on Applied Cryptography and Network Security*, pages 537–556. Springer, 2017.

[6] Berry Schoenmakers. A simple publicly verifiable secret sharing scheme and its application to electronic voting. In *Annual International Cryptology Conference*, pages 148–164. Springer, 1999.

[7] Ewa Syta, Philipp Jovanovic, Eleftherios Kokoris Kogias, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Michael J Fischer, and Bryan Ford. Scalable bias-resistant distributed randomness. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 444–460. IEEE, 2017.

[8] Miguel Castro, Barbara Liskov, et al. Practical byzantine fault tolerance. In *OSDI*, volume 99, pages 173–186, 1999.

[9] Douglas R Stinson and Reto Strobl. Provably secure distributed schnorr signatures and a (t, n) threshold scheme for implicit certificates. In *Australasian Conference on Information Security and Privacy*, pages 417–434. Springer, 2001.

[10] Philipp Schindler and Edgar Weippl. Practical continuous distributed randomness.

[11] Timo Hanke, Mahnush Movahedi, and Dominic Williams. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548*, 2018.

[12] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 514–532. Springer, 2001.

[13] Randao.org. Randao: Verifiable random number generation. 2017.

[14] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[15] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE transactions on information theory*, 31(4):469–472, 1985.

[16] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 223–238. Springer, 1999.

[17] Craig Gentry and Dan Boneh. *A fully homomorphic encryption scheme*, volume 20. Stanford University Stanford, 2009.

[18] Shahzadi Farah, Younas Javed, Azra Shamim, and Tabassam Nawaz. An experimental study on performance evaluation of asymmetric encryption algorithms. In *Recent Advances in Information Science, Proceeding of the 3rd European Conf. of Computer Science,(EECS-12)*, pages 121–124, 2012.

[19] Rounak Sinha, Hemant Kumar Srivastava, and Sumita Gupta. Performance based comparison study of rsa and elliptic curve cryptography. *International Journal of Scientific & Engineering Research*, 4(5):720–725, 2013.

[20] Yevgeniy Dodis and Aleksandr Yampolskiy. A verifiable random function with short proofs and keys. In *International Workshop on Public Key Cryptography*, pages 416–431. Springer, 2005.